STATEMENT OF WORK

PRODUCT 003

Safi Continuous AI Builder

GitHub Actions Test / Triage / Fix Orchestrator — Canonical Coffee Shop Reference

6 HOURS MAXIMUM — DELIVER OR NOTHING

1. FINISHED PRODUCT

Build a GitHub Actions-based autonomous development loop that can repeatedly evolve a working OpenAPI + SQLite backend from a plain-English TASK, using an advanced headless AI coding agent, while an independent protected Harness determines whether each change is actually correct.

```
TASK.md
   ↓
AI BUILD
   ↓
TEST
   ↓ fail
TRIAGE
   ↓
MINIMAL FIX
   ↓
TARGETED VERIFY
   ↓
FULL REGRESSION
   ↓ pass
INDEPENDENT VERIFY
   ↓
GOOD GIT STATE → AWS DEPLOY → DONE
```

AI BUILDS. THE HARNESS JUDGES.

- The Canonical Coffee Shop is the reference laboratory, not hard-coded into the orchestrator.
- The builder must perform multiple Test → Triage → Fix cycles automatically without a developer manually editing code between cycles.
- Every accepted change becomes a new known-good Git state. Failed candidate changes are discarded or superseded cleanly.
- OpenAPI remains the API contract; SQLite keeps the reference backend self-contained.
- Every eligible OpenAPI operation must remain MCP-ready through a generic validation/adapter test. A production MCP Gateway is out of scope.

2. STARTING POINT / PRECONDITIONS

- Company Git/GitHub repository and GitHub Actions are available.
- An existing AWS deployment path for a small service is available. Do not invent new AWS infrastructure in this Product.
- Use one existing supported runtime/language for the Coffee Shop; do not build multi-language support.
- Use one headless advanced AI coding engine for Product 003. Default: Codex CLI or the company's selected equivalent, invoked behind a tiny Agent Driver.
- Dyad-style interactive development remains compatible with the same Git repository and AI_RULES.md, but Dyad itself is not required to run headless inside GitHub Actions.
- Reasonable AI/API usage is reimbursed. Optimize for a finished Product, not token minimization.

3. CANONICAL COFFEE SHOP REFERENCE BACKEND

Use one small but non-trivial backend so the continuous builder can be judged objectively.

TABLE	BASELINE FIELDS
`customers`	`id, name, email, created_at`
`menu_items`	`id, sku, name, description, category, price_cents, active`
`orders`	`id, customer_id, status, total_cents, created_at, updated_at`
`order_items`	`id, order_id, menu_item_id, quantity, unit_price_cents, line_total_cents`

Baseline operations:

- listMenuItems
- getMenuItem
- createCustomer
- getCustomer
- createOrder

Proprietary and Copyright 2026 B. Gopinath

- getOrder
- listCustomerOrders
- updateOrderStatus

Business truth belongs on the server: createOrder reads prices from SQLite, computes totals, writes order + items transactionally, and enforces valid order-state transitions.

4. REPOSITORY CONTRACT

```
coffee-shop/
  AI_RULES.md
  TASK.md                         # immutable during one run
  openapi.yaml
  schema.sql
  seed.sql
  src/
  tests/generated/                # AI may create/change
  harness/acceptance/             # PROTECTED; AI may not change
  tools/agent_driver/
  tools/mcp_readiness/
  artifacts/cycles/               # triage, diffs, logs, results
  .github/workflows/
      safi-continuous-builder.yml
```

- AI_RULES.md defines stack, architecture, coding constraints and protected paths.
- TASK.md contains exactly one requested Product change and is read-only to the coding agent during the run.
- harness/acceptance/** is the independent judge. Any candidate patch touching it must fail immediately.
- tests/generated/** may be created or improved by AI, but generated tests never replace protected acceptance tests.
- The orchestrator itself must contain zero coffee/menu/order-specific logic.

5. THE TEST → TRIAGE → FIX CYCLE

The cycle is the Product. AI must not simply 'keep trying.' Every failure is first triaged into a structured engineering diagnosis, then fixed minimally.

```
1. RUN TESTS
2. IF FAIL → CAPTURE FAILURE BUNDLE
3. TRIAGE AGENT → triage.json
4. FIX AGENT → smallest candidate patch
5. VERIFY protected files unchanged
6. RUN TARGETED TEST(S)
7. IF PASS → RUN FULL REGRESSION
8. IF PASS → INDEPENDENT VERIFIER
9. IF APPROVED → COMMIT NEW GOOD STATE
10. ELSE → NEXT TRIAGE CYCLE
```

6. FAILURE BUNDLE AND STRUCTURED TRIAGE

For every failed cycle, the Triage role receives engineering evidence, not vague prose:

- TASK.md and AI_RULES.md
- current OpenAPI, schema and relevant source
- exact failing command
- stdout / stderr
- failing test name(s)
- expected vs actual result
- relevant logs
- current candidate Git diff
- prior cycle summaries

```
{
  "failure_class": "BUSINESS_LOGIC",
  "root_cause": "Size price adjustment is not applied to unit price",
  "evidence": ["expected 650; actual 550"],
  "files_likely_involved": ["src/orders.*"],
  "files_that_should_not_change": ["TASK.md", "harness/acceptance/**"],
  "fix_strategy": "Apply adjustment before line-total calculation",
  "confidence": "high"
}
```

Canonical failure classes:

```
BUILD, OPENAPI_CONTRACT, DATABASE_SCHEMA, MIGRATION, BUSINESS_LOGIC, STATE_TRANSITION, INPUT_VALIDATION,
OUTPUT_VALIDATION, REGRESSION, MCP_READINESS, DEPLOYMENT, TEST_INFRASTRUCTURE, TRANSIENT, SPEC_AMBIGUITY
```

SPEC_AMBIGUITY IS A VALID STOP CONDITION. DO NOT INVENT REQUIREMENTS.

Proprietary and Copyright 2026 B. Gopinath

7. FIX RULES

- Apply the smallest change that addresses the triaged root cause.
- Do not refactor unrelated code.
- Do not weaken validation or change success criteria.
- Do not delete or modify protected acceptance tests.
- Do not modify TASK.md.
- Do not hard-code test values merely to make a test pass.
- If the targeted test fails, return to Triage before another fix.
- Only after targeted PASS run the full regression suite.

8. GIT GOOD-STATE DISCIPLINE

```
GOOD-000 (baseline)
   ├─ candidate-01 → FAIL → discard / supersede
   ├─ candidate-02 → PASS → verifier PASS
   ↓
GOOD-001
   ├─ next TASK / candidate...

Every accepted change becomes a new known-good commit.
```

A failed candidate must never silently become the base for the next task. Git is the recovery mechanism and the audit trail.

9. AI ROLES AND AGENT DRIVER

Use one coding engine, but invoke it in controlled roles. The Harness owns the loop.

ROLE	INPUT	REQUIRED OUTPUT
Builder	TASK + repo	Plan + implementation candidate.
Triage	Failure bundle	Structured triage.json; no code edits.
Fixer	triage.json + repo	Minimal patch only.
Verifier	TASK + diff + passing tests	Independent review for shortcuts, hard-coding, weakened validation, scope expansion or test-gaming.

Agent Driver interface (minimal):

```
build(task, repo)
triage(failure_bundle, repo)
fix(triage, repo)
verify(task, repo, diff, test_results)
```

Product 003 implements only one real driver. Do not build multi-agent-engine support; keep the interface small so the engine can be replaced later.

10. MAXIMUM CYCLE POLICY

- Maximum 5 Test/Triage/Fix cycles per TASK.
- PASS requires targeted tests, full regression and independent verifier approval.
- After Cycle 5 failure, stop automatically and produce a Failure Report.
- On SPEC_AMBIGUITY, stop immediately with BLOCKED rather than consuming more cycles.
- On transient infrastructure failures, retry only the infrastructure step; do not ask AI to change application code.

11. MCP-READY DEFINITION FOR PRODUCT 003

MCP-ready is a validation property, not a production MCP server in this SOW.

- OpenAPI 3.1.x is the canonical contract for the reference backend.
- Every eligible operation has a unique operationId, useful description, request schema and response schema.
- A generic mcp_readiness tool converts/validates each eligible OpenAPI operation as an MCP-style Tool definition without Coffee Shop-specific code.
- The harness must fail if an iteration leaves an operation incompatible with the generic MCP adapter.
- Building the shared production MCP Gateway is a later Product.

12. THREE SEQUENTIAL COFFEE SHOP TASKS — REQUIRED PROOF

The Product is not proven by one lucky generation. Within the six-hour build, demonstrate three successive changes, each starting from the previously accepted GOOD state.

TASK	CHANGE	KEY ACCEPTANCE
1	**Add drink sizes**	SMALL/MEDIUM/LARGE for drinks; Medium +$0.50, Large +$1.00; food requires no size; server computes final unit price; OpenAPI/schema/code/tests remain synchronized.
2	**Add milk choices**	Drink orders may specify whole, skim, oat or almond; food unaffected; behavior added without changing orchestrator/harness logic; all Task 1 regressions still pass.

Proprietary and Copyright 2026 B. Gopinath

3	**Cancellation rule**	Orders may be cancelled only before PREPARING. Invalid transitions fail predictably. All prior ordering/size/milk behavior still passes.

AFTER TASK 3: ALL BASELINE + TASK 1 + TASK 2 + TASK 3 ACCEPTANCE TESTS MUST PASS.

13. HOW TO BUILD IT — AI-FIRST SIX-HOUR PLAN

TIME	PHASE	AI-FIRST EXECUTION	EXIT GATE
0:00–0:30	**Inspect + Freeze**	Give AI existing Git/AWS conventions and SOW. Select one runtime, one headless coding engine, existing deploy target and baseline repo structure. Create/confirm AI_RULES.md. Do not research alternate stacks.	Stack and paths frozen.
0:30–1:15	**Baseline Coffee Shop**	Use AI to create/complete working OpenAPI + SQLite Coffee Shop baseline and seed data. Establish a known-good baseline commit. Keep backend self-contained.	GOOD-000 passes baseline.
1:15–2:00	**Protected Harness**	Build protected acceptance runner, OpenAPI contract checks, SQLite reset, MCP-readiness check, protected-file guard, failure-bundle generator and targeted/full test commands.	Harness can judge independently.
2:00–2:45	**Agent Roles**	Implement minimal Agent Driver plus Builder/Triage/Fix/Verifier prompts. Triage must emit structured JSON. Add cycle artifact capture and max-5 policy.	One local Test/Triage/Fix cycle works.
2:45–3:30	**GitHub Actions Loop**	Implement reusable workflow: checkout good state → build candidate → tests → failure bundle → triage → fix → targeted verify → full regression → verifier → commit/stop. Persist cycle artifacts.	Headless loop works in Actions.
3:30–4:10	**Task 1**	Run Add Drink Sizes end-to-end. Do not manually edit code during repair cycles. Observe/repair only orchestrator/harness defects.	Task 1 reaches GOOD-001.
4:10–4:45	**Task 2**	Run Add Milk Choices from GOOD-001. Existing tests must remain green. No harness/orchestrator Coffee Shop special cases.	Task 2 reaches GOOD-002.
4:45–5:20	**Task 3 + Dogfood**	A second team member writes/triggers the Cancellation Rule TASK in plain English. Builder must evolve from GOOD-002 without manual code edits. User verifies resulting API behavior.	Task 3 reaches GOOD-003.
5:20–5:40	**Deploy + Live Smoke**	Deploy GOOD-003 using existing AWS path. Run live OpenAPI smoke tests and MCP-readiness check against the delivered build where applicable.	Live endpoint works.
5:40–6:00	**Independent Review**	Give AI full diff, cycle artifacts and test results. Fix only SOW-blocking defects. Confirm protected files untouched; commit/push final; produce concise run report.	Delivered by Hour 6.

14. REQUIRED CYCLE ARTIFACTS

```
artifacts/cycles/task-01/cycle-01/
  failure-bundle.json
  triage.json
  candidate.diff
  targeted-test.txt
  regression-test.txt
  verifier.json

Repeat per cycle/task.
```

- Do not store hidden chain-of-thought. Store only engineering evidence: inputs, diagnosis, patch, commands and results.
- Final run report includes task, number of cycles, PASS/FAIL/BLOCKED, elapsed action time, AI/API usage if available, and final good commit.

15. ACCEPTANCE TESTS

ID	TEST	PASS CONDITION
A1	**Baseline**	Fresh checkout initializes SQLite and all baseline Coffee Shop acceptance tests pass.
A2	**Protected Judge**	Attempted edit to TASK.md or harness/acceptance/** is automatically rejected.
A3	**Structured Triage**	A deliberately failing change produces machine-readable triage.json with class, root cause, evidence and fix strategy.
A4	**Minimal Repair**	At least one failed candidate is repaired automatically through Triage → Fix → targeted test → regression without manual code editing.
A5	**Rollback/Good State**	Failed candidates do not become the base; accepted PASS becomes a new known-good Git commit.
A6	**Task 1**	Drink Sizes reaches PASS including OpenAPI, SQLite, pricing and regression tests.
A7	**Task 2**	Milk Choices reaches PASS from Task 1 state; Task 1 and baseline tests remain green.
A8	**Task 3**	Cancellation Rule reaches PASS from Task 2 state; all previous tests remain green.
A9	**Independent Verify**	Passing candidate receives separate verifier approval and no test-gaming/hard-coded shortcut is detected.
A10	**MCP Readiness**	All eligible OpenAPI operations still convert/validate generically as MCP-ready after Task 3.
A11	**GitHub Actions**	At least one complete task is built through the actual GitHub Actions loop, not only local scripts.

Proprietary and Copyright 2026 B. Gopinath

A12	**Dogfood**	A team member other than the builder authors/triggers one TASK and verifies the delivered API behavior.
A13	**AWS Delivery**	Final GOOD state is deployed through the existing AWS path and live smoke tests pass.

16. DELIVERABLES BY HOUR 6

- Working Canonical Coffee Shop OpenAPI + SQLite baseline.
- GitHub Actions continuous builder workflow.
- One operational headless AI Agent Driver with Builder, Triage, Fix and Verifier roles.
- Protected acceptance harness with targeted + full regression testing.
- OpenAPI contract validation and generic MCP-readiness validation.
- Automatic failure-bundle and structured triage artifacts.
- Maximum-5-cycle control, BLOCKED behavior for spec ambiguity, and clean failure report.
- Git known-good state discipline and candidate rollback/supersession.
- Three sequential Coffee Shop changes completed and all cumulative regressions green.
- Final known-good version deployed on existing AWS infrastructure.
- Code/config committed and pushed to company Git.
- Short README: architecture, run locally, trigger TASK, inspect cycle artifacts, add acceptance test, deploy.

17. OUT OF SCOPE

- Production MCP Gateway.
- Security/authentication platform work beyond existing environment defaults.
- Multiple AI coding engines or engine benchmarking.
- Multiple programming languages/runtimes.
- Kubernetes or new AWS infrastructure.
- Frontend/UI generation in Product 003. Keep repository structure Dyad-compatible for future frontend work.
- Automatic generation of acceptance criteria from vague business requirements without human ownership.
- Infinite repair loops or autonomous weakening of requirements/tests.
- Generalizing the orchestrator to every possible repository type; prove it rigorously on the canonical backend first.

18. DEFINITION OF DONE

```
TASK → GITHUB ACTION → AI BUILD
TEST → TRIAGE → MINIMAL FIX → TARGETED VERIFY → FULL REGRESSION
PASS → INDEPENDENT VERIFY → NEW GOOD GIT STATE → AWS DEPLOY
PROOF: 3 SUCCESSIVE COFFEE SHOP TASKS; CUMULATIVE REGRESSION PASS
```

If a human must manually edit application code to rescue the three required tasks, the Continuous Builder is NOT DONE.

19. SIX-HOUR RULE

The six hours include baseline creation, Harness, agent roles, GitHub Actions loop, automatic Test/Triage/Fix repair, three sequential changes, dogfooding, AWS deployment, final verification, Git delivery and README.

NO SEVENTH HOUR. NO “ALMOST DONE.” HARNESS PASS OR PRODUCT FAILED.

Proprietary and Copyright 2026 B. Gopinath